

PRACTICAL 6

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization

Code:

```
main.c
1 // Online C compiler to run C program online
2 #include <stdio.h>
3 #include <pthread.h>
4 #include <semaphore.h>
5 #include <unistd.h>
6
7 #define N 5 // Number of philosophers
8
9 sem_t chopsticks[N]; // Semaphores for
    chopsticks
10
11 void *philosopher(void *num) {
12     int id = *(int *)num;
13
14     while (1) {
15         printf("Philosopher %d is thinking.\n",
            id);
16         sleep(1); // Thinking
17
18         // Pick up chopsticks
19         if (id % 2 == 0) {
20             sem_wait(&chopsticks[id]); // Pick
```

```
main.c
20         sem_wait(&chopsticks[id]); // Pick
            left chopstick
21         sem_wait(&chopsticks[(id + 1) % N]);
            // Pick right chopstick
22     } else {
23         sem_wait(&chopsticks[(id + 1) % N]);
            // Pick right chopstick first
24         sem_wait(&chopsticks[id]); // Then
            pick left chopstick
25     }
26
27     printf("Philosopher %d is eating.\n", id
        );
28     sleep(2); // Eating
29
30     // Put down chopsticks
31     sem_post(&chopsticks[id]);
32     sem_post(&chopsticks[(id + 1) % N]);
33
34     printf("Philosopher %d finished eating
        and is thinking again.\n", id);
35     sleep(1);
```

```
main.c
35     sleep(1);
36 }
37 }
38
39 int main() {
40     pthread_t philo[N];
41     int phil_id[N];
42
43     // Initialize semaphores (chopsticks)
44     for (int i = 0; i < N; i++)
45         sem_init(&chopsticks[i], 0, 1);
46
47     // Create philosopher threads
48     for (int i = 0; i < N; i++) {
49         phil_id[i] = i;
50         pthread_create(&philo[i], NULL,
51             philosopher, &phil_id[i]);
52     }
53
54     // Join threads (infinite loop, so it won't
55     // actually exit)
56     while (1)
57         continue;
58 }
```

```
51     }
52
53     // Join threads (infinite loop, so it won't
54     // actually exit)
55     for (int i = 0; i < N; i++)
56         pthread_join(philo[i], NULL);
57
58     return 0;
59 }
```

Output :

```
Output Clear
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is eating.
Philosopher 3 is eating.
Philosopher 3 finished eating and is thinking again.
Philosopher 1 is eating.
Philosopher 0 finished eating and is thinking again.
Philosopher 4 is eating.
Philosopher 3 is thinking.
Philosopher 0 is thinking.
Philosopher 1 finished eating and is thinking again.
Philosopher 4 finished eating and is thinking again.
Philosopher 2 is eating.
Philosopher 0 is eating.
Philosopher 1 is thinking.
Philosopher 4 is thinking.
Philosopher 2 finished eating and is thinking again.
Philosopher 1 is eating.
Philosopher 0 finished eating and is thinking again.
```